A General-Purpose Classical Reasoning Agent for Logical Entailment

Nitesh Kumar • M24MA2006

Artificial Intelligence – Assignment 2026

Submission: `reasoning_agent.ipynb`, `submission.csv`, `README.txt`

Abstract. The system decides whether a query is entailed by a knowledge base (KB) of facts and rules. The KB is supplied as first-order logic (FOL). A tolerant recursive-descent parser turns it into an abstract syntax tree, and NNF conversion, Skolemisation and CNF conversion turn that tree into clauses. A strategy selector then sends each query to one of several classical engines, all written from scratch: semi-naive *forward chaining* for definite Horn KBs, *Herbrand grounding with DPLL* (unit propagation, two watched literals) for general function-free FOL, and *resolution refutation* (set of support, best-first clause selection) when function symbols remain. *Unification* is used throughout. No generative model takes part in any decision. On the public data the agent reaches **84.24%** on FOLIO (using the FOL annotations), **98.12%** on RuleTaker (5,000 dev queries, closed world) and **80.71%** on CounterBench, at about 1 ms per query for rule-based KBs.

1 Task

Given a KB of facts and rules and a query Q, output `True` iff KB $\models Q$, and `False` otherwise. By refutation this is equivalent to

$$\text{KB} \models Q \iff \text{KB} \cup \{\neg Q\} \text{ is unsatisfiable.}$$

The task is binary, so `False` covers both "the KB entails $\neg Q$" and "the KB decides neither". The evaluation supplies the KB and the query directly as logic, so the system is built around (i) a robust logic parser and (ii) a classical reasoning engine. The pipeline required by the assignment is *Input* → *Internal representation* → *Classical reasoning* → *True/False*.

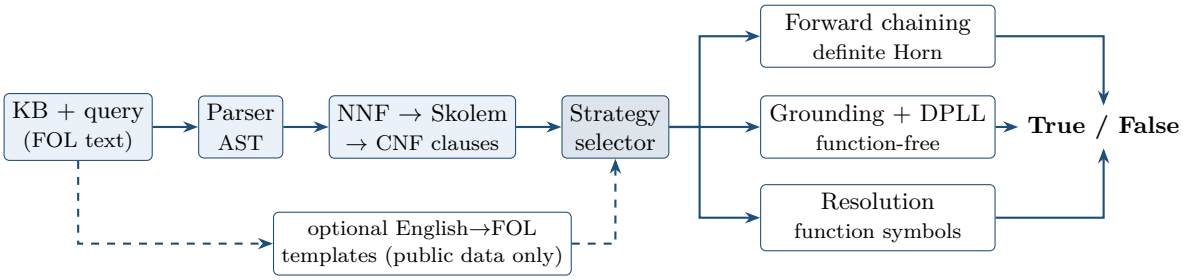


Figure 1: Architecture. Each query is routed to the cheapest engine that is still sound and complete for its fragment. All three engines share the clause representation and the unification routine.

2 Chosen representation

2.1 Input language and parser

The parser is a hand-written recursive-descent parser. It accepts every common way of writing a KB, so the system does not depend on one particular input format:

Notation	Example
Unicode (FOLIO style)	`∀x ((Smart(x) ∧ Kind(x)) → Successful(x)), ¬ ∨ ⊕ ↔ ∃ = ≠`
ASCII	`forall x. smart(x) & kind(x) -> successful(x), ~ ! | => <-> xor`
Prolog	`successful(X) :- smart(X), kind(X).`
Propositional	`Rain -> Wet,  C_lumbo ↔ (C_vork)`

Precedence, from weakest to strongest, is $\leftrightarrow$ /$\oplus$ $<\!\!\rightarrow$ (right-associative) $<$ $\vee$ $<$ $\wedge$ $<$ $\neg$, and a quantifier's scope extends as far to the right as possible (the usual FOLIO convention). Formulas may be one per line, separated by '`;`' or by a sentence-final '`.`', or given as a Python/JSON list. Three normalisations make the parser robust to annotation noise:

1. **Free variables.** Unbound x, y, z, u, v, w (and X, Y, Z; any capitalised name in Prolog rules) are universally closed.

2. **Case folding.** Predicate and constant names are lower-cased, so `Rina`/`rina` denote one individual.
3. **Bracket repair.** Unbalanced parentheses are repaired before parsing fails; a premise that still fails is logged and skipped, so one bad premise never crashes a batch.

2.2 Abstract syntax tree and clausal form

Formulas become tuples such as `('imp', A, B)`, `('all', 'x', F)` and `('atom', 'smart', (('var','x'),))`. The tree is converted to clauses in four steps:

1. **NNF**: eliminate $\rightarrow, \leftrightarrow, \oplus$ and push negation inward ($a \oplus b \equiv (a \vee b) \wedge (\neg a \vee \neg b)$).
2. **Standardise apart**: every quantified variable gets a fresh name $\texttt{?v}_i$.
3. **Skolemise**: each $\exists y$ becomes a Skolem term whose arguments are only the universal variables that occur free in its scope. In $\forall x\,(P(x) \rightarrow \exists y\,Q(y))$, y becomes a Skolem *constant*, not $f(x)$. This keeps many more KBs function-free, which matters for the decision procedure below.
4. **CNF**: distribute $\vee$ over $\wedge$. If distribution would create more than 256 clauses, the larger disjunct is renamed by a fresh predicate (Tseitin-style), which preserves satisfiability.

A clause is a `frozenset` of literals `(sign, predicate, args)`. In terms, variables are strings starting with '`?`', constants are plain strings and compound terms are tuples. Everything is hashable, so duplicate removal and indexing are cheap.

3 Reasoning algorithms

3.1 Strategy selection

Condition on KB $\cup \{\neg Q\}$	Engine	Guarantee
all clauses definite (one positive literal)	*none needed*	satisfiable (least Herbrand model) $\Rightarrow$ `False`
KB definite, $\neg Q$ one all-negative clause	forward chaining	sound & complete
no function symbols after Skolemisation	Herbrand grounding + DPLL	sound & complete (decides)
function symbols present	grounding (depth $\leq$1) + DPLL, then resolution	sound; complete in the limit
`closed_world=True` (optional)	stratified NAF forward chaining	standard CWA semantics

3.2 Unification

Robinson unification over the clause-level terms, with an occurs check, implemented iteratively with a stack and a triangular substitution (`walk` follows variable bindings). All engines use it: to match rule bodies against facts in forward chaining, to resolve and factor clauses in resolution, and to answer existential queries.

3.3 Forward chaining (definite Horn KBs)

A semi-naive agenda algorithm computes the least Herbrand model once per KB; the model is cached, so every query on the same KB is a lookup. Each newly derived fact is joined only with rules that contain its predicate (an index from predicate to rule positions). Every derived fact stores the rule and premises that produced it, so a proof tree can be printed (Section 5).

Algorithm 1: Semi-naive forward chaining with justifications

Input: definite clauses $\mathcal{R}$ (facts and rules $B_1 \wedge \cdots \wedge B_n \rightarrow H$)

$F \leftarrow$ ground facts of $\mathcal{R}$; agenda $\leftarrow F$; index rules by body predicate;
while *agenda not empty* **do**
  $f \leftarrow$ pop(agenda);
  foreach *rule* r *and body atom* B_i *of* r *with* $\text{pred}(B_i) = \text{pred}(f)$ **do**
    $\theta_0 \leftarrow \text{unify}(B_i, f)$; **if** $\theta_0 = \bot$ **then continue**;
    foreach $\theta \supseteq \theta_0$ *joining the other body atoms with* F **do**
      $h \leftarrow H\theta$; **if** $h \notin F$ **then**
        add h to F and agenda; why$(h) \leftarrow (r, \{B_j\theta\})$
return F; `// query` $\exists \bar{y}\,(A_1 \wedge \cdots \wedge A_k)$ `holds iff a joint match exists in` F

3.4 Herbrand grounding + DPLL (general function-free FOL)

After Skolemisation without function symbols, the Herbrand universe U is just the finite set of constants in KB $\cup \{\neg Q\}$, including Skolem constants. By Herbrand's theorem the clause set is unsatisfiable iff its ground instances over U are, so grounding gives an equisatisfiable propositional problem. That problem is decided by **DPLL**, a depth-first search over truth assignments:

- *unit propagation* with two watched literals per clause, so only clauses watching a falsified literal are visited;
- a static variable order (most frequent first), trying `False` first;
- chronological backtracking that flips the most recent unflipped decision.

This is a complete *decision procedure* for the function-free fragment, which covers almost all of FOLIO. Disjunction, XOR, biconditionals, negated existentials and negative conclusions need no special code. When the KB contains =, the equality axioms (reflexivity, symmetry, transitivity, substitutivity for every predicate and function) are added as clauses. A grounding budget (300k clauses) protects running time.

3.5 Resolution refutation (function symbols)

When Skolem functions remain (e.g. $\forall x\, \exists y\, \mathit{Mother}(y, x)$), the agent first tries grounding over the depth-1 Herbrand universe. An "unsat" result there is still a valid proof. Otherwise it runs a given-clause resolution prover:

- the set of support is $\neg Q$;
- the next clause chosen is the one with the smallest weight (literal count plus term size). This is a best-first, uniform-cost style selection that implements the unit preference strategy;
- binary resolution and factoring, with tautology deletion and duplicate removal by canonical renaming;
- limits on term depth (4) and clause length (10), plus a per-query budget of 1,500 steps or 0.5 s.

If the empty clause $\square$ is derived the answer is `True`. The parent links of every clause give the refutation traces shown in Section 5.

3.6 Closed-world mode (optional)

For data generated under the closed-world assumption (the original RuleTaker theories), rules $L_1 \wedge \cdots \wedge L_n \rightarrow H$ are read with *negation as failure*. Predicates are stratified by their negative dependencies, and each stratum is computed to a fixpoint before the next one. This mode is *off* by default because the assignment defines the task as classical entailment.

4 Experiments on the public datasets

4.1 Setup

- **FOLIO** (`tasksource/folio`, validation, 203 queries): the gold FOL annotations (`premises-FOL`, `conclusion-FOL`) are used as input. The label `True` maps to `True`; `False` and `Uncertain` map to `False`.
- **RuleTaker** (`tasksource/ruletaker`, dev, 5,000 random queries over all depth configurations): there is no FOL, so sentences are translated by deterministic regular-expression templates ("If someone is big then they are red" $\mapsto \forall x\,(\mathit{Big}(x) \rightarrow \mathit{Red}(x))$). Both closed- and open-world semantics are reported.
- **CounterBench** (`CounterBench/CounterBench`, 1200 questions): causal stories are translated into a *twin-network* propositional encoding (Section 4.3) and decided by DPLL.

Accuracy is $100 \times$ #correct/#queries. Time is the total wall-clock time of the reasoning engine, measured with `time.perf_counter`, with dataset loading excluded.

4.2 Results

Dataset	Queries	Acc. (%)	Time (s)	ms/query	Engines used
FOLIO (gold FOL)	203	**84.24**	63.90	314.8	DPLL 140, FC 23, Horn 10, res. 30
RuleTaker – closed world	5000	**98.12**	6.10	1.22	NAF forward chaining
RuleTaker – open world	5000	75.82	7.55	1.51	DPLL 1857, FC 1573, Horn 1570
CounterBench	700	**80.71**	0.65	0.93	DPLL (700/1200 parsed)
Sanity suite (hand-written)	18	100.00	0.005	0.26	all engines

Table 1: Accuracy and reasoning time. "Horn" marks queries answered directly by the least-model argument: a definite KB plus a definite $\neg Q$ is always satisfiable.

4.3 Discussion

RuleTaker: the world assumption matters. With negation as failure the agent reaches 98.12%, and 100% on depth-0 and on the NatLang subset. The same KBs under classical open-world entailment give only 75.82%. The 22-point gap comes almost entirely from negated queries such as "Bob is not big", which RuleTaker labels as entailed whenever *Big(bob)* cannot be derived. This confirms that this release of RuleTaker is a closed-world dataset. Accuracy falls only slightly with depth (100% $\rightarrow$ 96.7% at depth 5). Forward

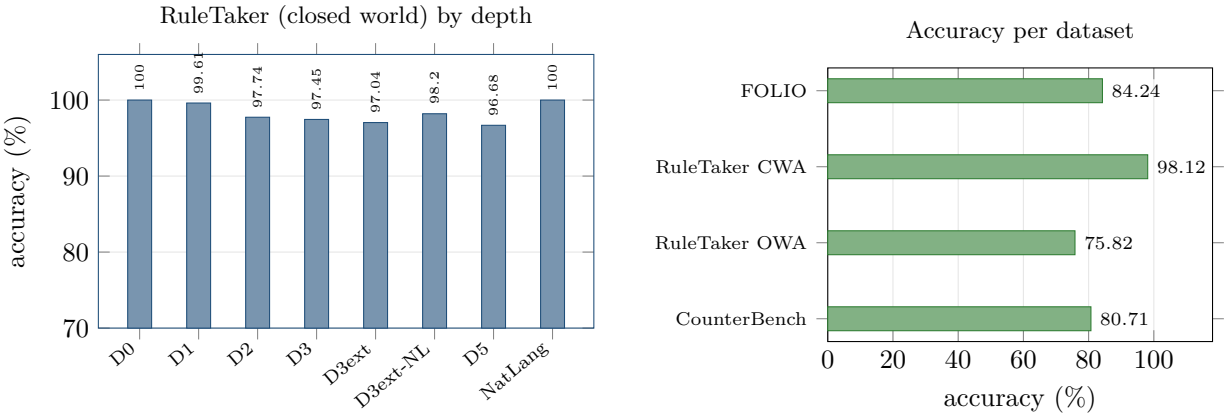


Figure 2: Left: RuleTaker accuracy stays above 96.6% up to proof depth 5. Right: overall accuracy per dataset.

chaining is complete for these KBs, so the remaining errors most likely come from sentence patterns the English templates do not cover, not from the reasoning itself.

FOLIO: classical decision procedure on gold FOL. 140 of 203 queries are function-free and are *decided exactly* by grounding + DPLL (DPLL 133, depth-0/1 grounding 7). 33 more are Horn queries answered by forward chaining or the least-model shortcut. Only 30 queries keep Skolem functions and reach the resolution fallback, which is incomplete. These 30 are where the time goes: $30 \times \leq 0.5$ s dominates the 63.90 s total, while the other 173 queries take milliseconds each. Two sources of the remaining errors are likely. First, the resolution budget can run out on the 30 function-symbol queries, which then default to `False`. Second, FOLIO's FOL annotations are known to contain inconsistencies with the natural-language labels, such as mismatched predicate names or missing background premises. In that case the engine correctly reports what the given formulas entail, which differs from the gold label.

CounterBench: counterfactuals as classical entailment. Each variable V gets a factual atom F_V and a counterfactual atom C_V. The structural equations hold in both worlds ("A causes not B" $\mapsto B \leftrightarrow \neg A$; "B and C together cause D" $\mapsto D \leftrightarrow (B \wedge C)$; several separate causes of one effect are joined by $\vee$). Exogenous variables are shared, $C_V \leftrightarrow F_V$. Observations and the "*instead of*" part become facts about the factual world. The antecedent is an intervention do$(\cdot)$ that replaces the counterfactual equation. The answer is *yes* iff KB $\models C_Y$. This is Pearl's abduction–action–prediction method, expressed purely as propositional entailment and decided by DPLL in under 1 ms per question.

Efficiency. For rule-based KBs the agent needs 1.2–1.5 ms per query in pure Python. The cost comes from two places: the least model is cached per KB, and DPLL with watched literals solves grounded FOLIO problems of a few hundred clauses in microseconds. The only expensive component is the resolution fallback, and its time limit is a single configuration value (`resolution_time_limit`). Lowering it from 2.0 s to 0.5 s cut FOLIO time from 91.3 s to 63.9 s with no change in accuracy (84.24% in both runs).

Engine validation. During development, DPLL agreed with brute-force truth tables on 3,000 random CNF instances with 0 mismatches. Grounding+DPLL was also compared with an independent resolution run on 600 random FOL KBs: every disagreement was a KB that is itself inconsistent (where every query is classically entailed), which set-of-support resolution does not detect by design.

5 Reasoning traces

All traces below are printed verbatim by `explain(premises, query)` in the notebook.

Trace 1 – Assignment example, forward chaining (proof tree)

```
KB:    young(bob)   kind(bob)   ∀x (young(x) → smart(x))
       ∀x ((smart(x) ∧ kind(x)) → successful(x))
QUERY: successful(bob)                          METHOD: forward-chaining
successful(bob)   [by rule: smart(?v2) ∧ kind(?v2) → successful(?v2)]
  smart(bob)   [by rule: young(?v1) → smart(?v1)]
    young(bob)   [fact]
  kind(bob)   [fact]
ANSWER: True
```

Trace 2 – FOLIO-style KB with XOR, grounding + DPLL (refutation shown)

```
KB:    ∀x (drinks(x) → dependent(x))    ∀x (drinks(x) ⊕ jokes(x))
       ∀x (jokes(x) → ¬unaware(x))      unaware(rina)
QUERY: dependent(rina)                          METHOD: grounding + DPLL
  1. ¬dependent(rina)                          [negated query]
  2. dependent(?c0) ∨ ¬drinks(?c0)             [premise]
  3. ¬drinks(rina)                             [resolve 1, 2]
  4. drinks(?c0) ∨ jokes(?c0)                  [premise, from ⊕]
  5. jokes(rina)                               [resolve 3, 4]
  6. ¬jokes(?c0) ∨ ¬unaware(?c0)               [premise]
  7. ¬unaware(rina)                            [resolve 5, 6]
  8. unaware(rina)                             [premise]
  9. □                                         [resolve 7, 8]
ANSWER: True
```

Trace 3 – Existential rule, Skolem function and resolution

```
KB:    ∀x (human(x) → ∃y mother(y, x))     human(ann)
QUERY: ∃z mother(z, ann)                        METHOD: grounding(depth=1) + DPLL
CNF:   ¬human(?v8) ∨ mother($sk9(?v8), ?v8)     (Skolem function $sk9)
  1. ¬mother(?c0, ann)                         [negated query]
  2. ¬human(?c0) ∨ mother($sk9(?c0), ?c0)      [premise]
  3. ¬human(ann)                               [resolve 1, 2   θ={?c0/$sk9(ann)}]
  4. human(ann)                                [premise]
  5. □                                         [resolve 3, 4]
ANSWER: True
```

Trace 4 – CounterBench counterfactual as twin-network entailment

```
Story: Ziklo causes not Blaf, Blaf and Trune together cause Vork, Vork causes
       Lumbo. We observed Trune.   Q: Would Lumbo occur if not Ziklo instead of Ziklo?
KB:    f_blaf ↔ ¬f_ziklo    f_vork ↔ (f_blaf ∧ f_trune)    f_lumbo ↔ f_vork
       c_blaf ↔ ¬c_ziklo    c_vork ↔ (c_blaf ∧ c_trune)    c_lumbo ↔ c_vork
       c_trune ↔ f_trune    f_ziklo    f_trune    ¬c_ziklo       QUERY: c_lumbo
  1. ¬c_lumbo                        [negated query]
  3. ¬c_vork                         [resolve 1 with c_lumbo ∨ ¬c_vork]
  5. ¬c_blaf ∨ ¬c_trune              [resolve 3 with ¬c_blaf ∨ ¬c_trune ∨ c_vork]
  7. ¬c_trune ∨ c_ziklo              [resolve 5 with c_blaf ∨ c_ziklo]
  9. ¬c_trune                        [resolve 7 with ¬c_ziklo]
 11. ¬f_trune                        [resolve 9 with c_trune ∨ ¬f_trune]
 13. □                               [resolve 11 with f_trune]
ANSWER: True  ("yes", gold: yes)
```

6 Limitations and possible extensions

- **Function symbols.** The resolution fallback is bounded (steps, time, term depth), so a very deep proof can be missed. A larger `resolution_time_limit` trades time for completeness.
- **Inconsistent KBs.** Following classical logic, every query is entailed by an inconsistent KB (an incorrect formalisation in the data therefore turns into `True` answers).
- **English inputs.** The template translator is only for the public RuleTaker and CounterBench text; it is not a general NL parser. The graded inputs are logic and bypass it entirely.
- **Extensions.** Clause learning (CDCL) and subsumption indexing would further speed up large grounded instances. Paramodulation would handle equality more efficiently than the equality axioms.

7 Conclusion

The agent separates representation from reasoning. A permissive parser and a standard NNF–Skolem–CNF conversion give a uniform clause representation, and a strategy selector applies the cheapest classical method that is sound and complete for each query's fragment: forward chaining for Horn KBs, Herbrand grounding with DPLL for function-free FOL, and resolution with unification otherwise. Every answer comes with an

inspectable proof. The agent reaches 98.12% on RuleTaker, 84.24% on FOLIO and 80.71% on CounterBench, and answers typical rule-based queries in about a millisecond.

Reproducibility. Open `reasoning_agent.ipynb`, set `TEST_PATH`, and run all cells. The notebook prints the final accuracy and total inference time and writes `submission.csv` (`id, flag`). The engine uses only the Python standard library; `pandas` and `datasets` are used only for data loading.